Two Ways to Recommend a Song

</> Code

Content-based vs. collaborative filtering, built and compared head to head

AUTHOR
Your Name

PUBLISHED
September 29, 2026

Interactive application

Open the Streamlit app

1 Introduction

I wanted to work on recommendation for this project because it's a genuinely different kind of machine-learning problem from anything else in my usual toolkit. Most tabular projects are supervised: there's a `y` the model is trying to predict, and a metric that scores how close its predictions come. Recommendation doesn't have that — there's no single "correct" next item — and once I started reading around the topic, I realized it splits into two genuinely different techniques depending on what data happens to be available:

- **Content-based** — no user data needed at all. Compare items by their own measurable properties (here, Spotify's audio features) and recommend whatever's most similar.
- **Collaborative filtering** — no item content needed at all. Learn from real user/item interaction history (here, KKBox's listening logs) instead.

Rather than pick one, I built and evaluated one working system of each kind, then put them side by side. That wasn't to crown a winner — they solve different problems, with different data requirements, so "which one wins" isn't really a coherent question to ask across both. I did it because comparing how each is built, evaluated, and interpreted taught me more about what recommendation actually is than building either one in isolation would have.

2 Part I — Content-Based Recommendation (Spotify)

The question I set out to answer here: given a track someone likes, can I recommend other tracks whose *audio* resembles it — not tracks other, similar listeners happened to play, since there's no listener data in this dataset at all. Every recommendation comes from Spotify's own audio-feature

measurements: danceability, energy, acousticness, and so on. I built this on Kaggle's Spotify Tracks Dataset.

2.1 Data

~114,000 tracks, roughly 1,000 sampled per genre across 114 genres, each with:

- **Audio features** (Spotify's own per-track measurements): `danceability`, `energy`, `loudness`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `valence` (musical positivity), `tempo`, `key` (one of 12 pitch classes), `mode` (major/minor), `time_signature`.
- **Metadata**: `track_name`, `artists`, `album_name`, `duration_ms`, `explicit`.
- `popularity` — a Spotify-computed score, and `track_genre` — a human-assigned genre tag. Neither is used as a recommender *input* feature (see Methodology) — both are reserved for evaluation.

▶ Code

```
89741 tracks, 113 genres
```

	count	unique	top	freq	mean	std	min	25%	50%	75%
danceability	89741.0	NaN	NaN	NaN	0.562166	0.176691	0.0	0.45	0.576	0.692
energy	89741.0	NaN	NaN	NaN	0.634458	0.256605	0.0	0.457	0.676	0.853
loudness	89741.0	NaN	NaN	NaN	-8.499004	5.22149	-49.531	-10.322	-7.185	-5.108
speechiness	89741.0	NaN	NaN	NaN	0.087442	0.113277	0.0	0.036	0.0489	0.0859
acousticness	89741.0	NaN	NaN	NaN	0.328289	0.338321	0.0	0.0171	0.188	0.625
instrumentalness	89741.0	NaN	NaN	NaN	0.173413	0.323848	0.0	0.0	0.000058	0.0976
liveness	89741.0	NaN	NaN	NaN	0.21697	0.194884	0.0	0.0982	0.132	0.279
valence	89741.0	NaN	NaN	NaN	0.469477	0.262864	0.0	0.249	0.457	0.682
tempo	89741.0	NaN	NaN	NaN	122.058316	30.117532	0.0	99.264	122.013	140.07
track_genre	89741	113	acoustic	1000	NaN	NaN	NaN	NaN	NaN	NaN
popularity	89741.0	NaN	NaN	NaN	33.198438	20.580824	0.0	19.0	33.0	49.0

2.1.1 Why I excluded `popularity` and `track_genre` from the feature space

I was tempted to throw every available column into the feature space and let the model sort it out, but two of them would have quietly broken the whole exercise. Including `popularity` as a

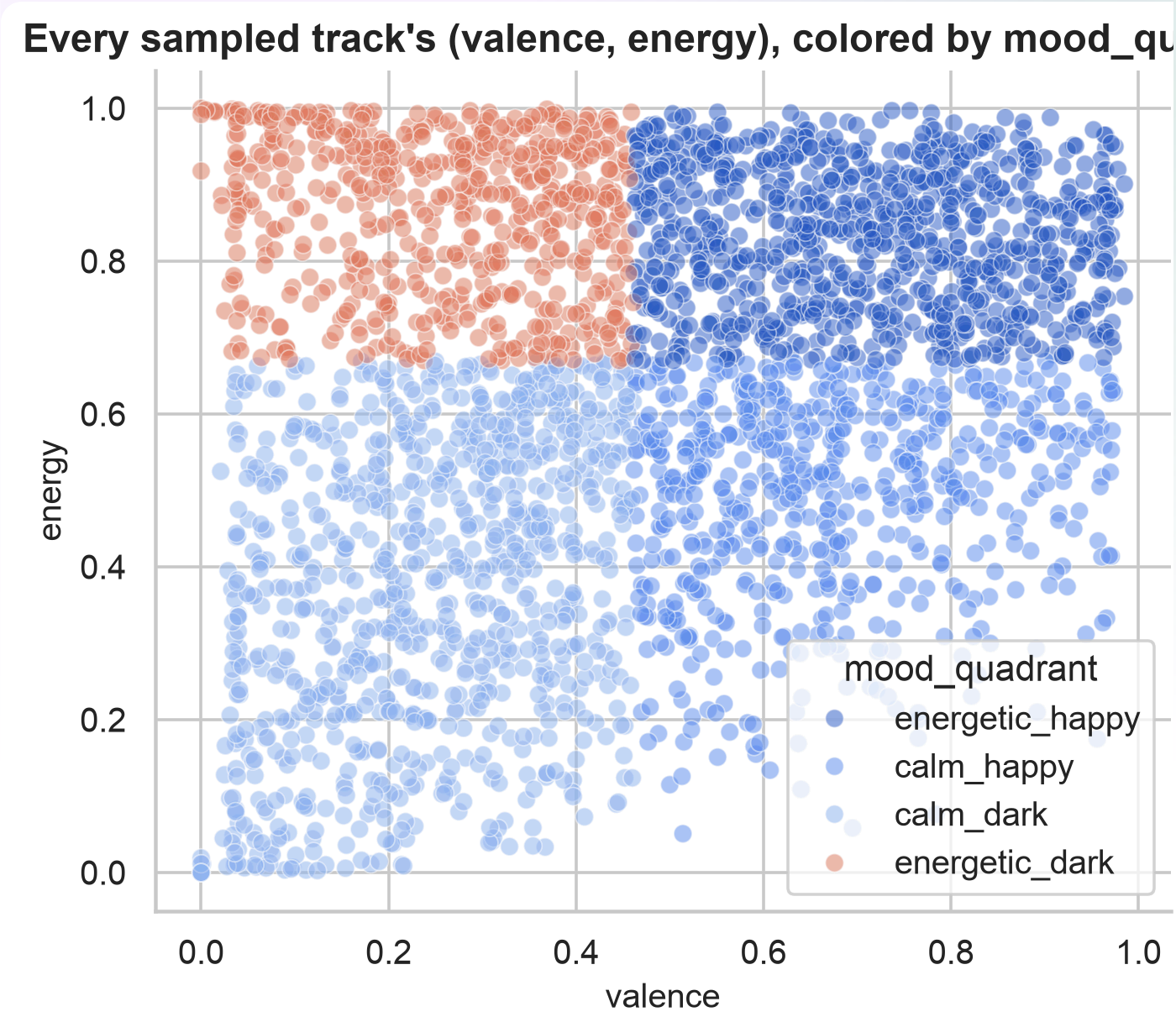
recommendation feature would make the system chase what's already trending rather than what actually sounds similar — the opposite of a useful recommender. Including `track_genre` directly would make the genre-matching evaluation below trivially easy (the recommender would just be reading the answer off the genre column), which would prove nothing about whether the *audio* similarity is meaningful. So I kept both out of the feature space entirely and only bring them back afterward, to score results.

2.2 Feature engineering

I engineered three features, detailed in `src/recommendation_showcase/content_based/features.py`:

- `key_sin` / `key_cos` — my first instinct was to leave `key` as a plain integer, until I remembered it's actually 12 pitch classes arranged in a circle: key 11 is one semitone from key 0, not "far away" from it the way a plain integer or a flat one-hot encoding would imply. Encoding it as a point on a circle (sine and cosine of the angle) preserves that adjacency — the standard trick for any cyclical feature, and the same idea I'd reach for with week-of-year or hour-of-day.
- `duration_min` — `duration_ms / 60000`, a pure readability transform, nothing more.
- `mood_quadrant` — buckets each track into one of four moods (energetic/calm x happy/dark) from Russell's circumplex model of affect, using training-data medians of `valence` / `energy` as the quadrant boundaries. The classifier can already learn this distinction from the raw floats directly, so this feature doesn't add predictive power — I added it purely for human legibility in the notebook and app, where "energetic_happy" reads far more naturally than two decimals.

▶ Code



The four color clusters aren't perfectly separated blobs — they shade into each other near the median cutoffs, which makes sense since mood is genuinely continuous, not four discrete buckets. What this chart tells me is that the quadrant labels are a reasonable simplification for reading a chart or scanning a recommendation list, but the actual distance calculations underneath still use the raw `valence` / `energy` floats, not this coarser summary.

2.3 Methodology

2.3.1 The recommender: weighted nearest-neighbor search

`ContentRecommender` (`content_based/model.py`) standardizes every audio feature first — scale matters directly for distance in a way it never would for a tree-based model, since a raw feature with a wide numeric range (like `loudness`, roughly -60 to 0) would otherwise dominate the distance calculation over a feature that's actually more musically meaningful but happens to live on a 0-1 scale (like `danceability`). After scaling, it finds the `k` nearest tracks by Euclidean distance in that feature space. I tried three optional ways to reshape that distance metric before the search runs, and evaluated each below rather than assuming any of them would help:

- **Importance-weighted** — fit a genre classifier on the same features and use its learned feature importances to *weight* the distance metric: a feature the classifier leans on to separate genres counts more toward "closeness."
- **PCA** — project onto the top 10 principal components before searching. With only ~25 features to begin with, this is more a test of whether discarding lower-variance components denoises the space than a genuine dimensionality-curse fix (PCA's usual selling point matters more at hundreds or thousands of dimensions than it does here).
- **NCA** (Neighborhood Components Analysis) — learn a linear projection via gradient descent that directly maximizes KNN accuracy under the genre labels, a metric *learned specifically for* nearest-neighbor search rather than borrowed from elsewhere. Its cost scales roughly quadratically with row count and gets OOM-killed on this project's full ~72k-row training split (I verified this directly — the process gets killed by the OS for exhausting memory), so it's fit on a 5,000-row subsample instead, then applied to the full catalog.

2.3.2 Evaluating a recommender with no user-feedback data: precision@k

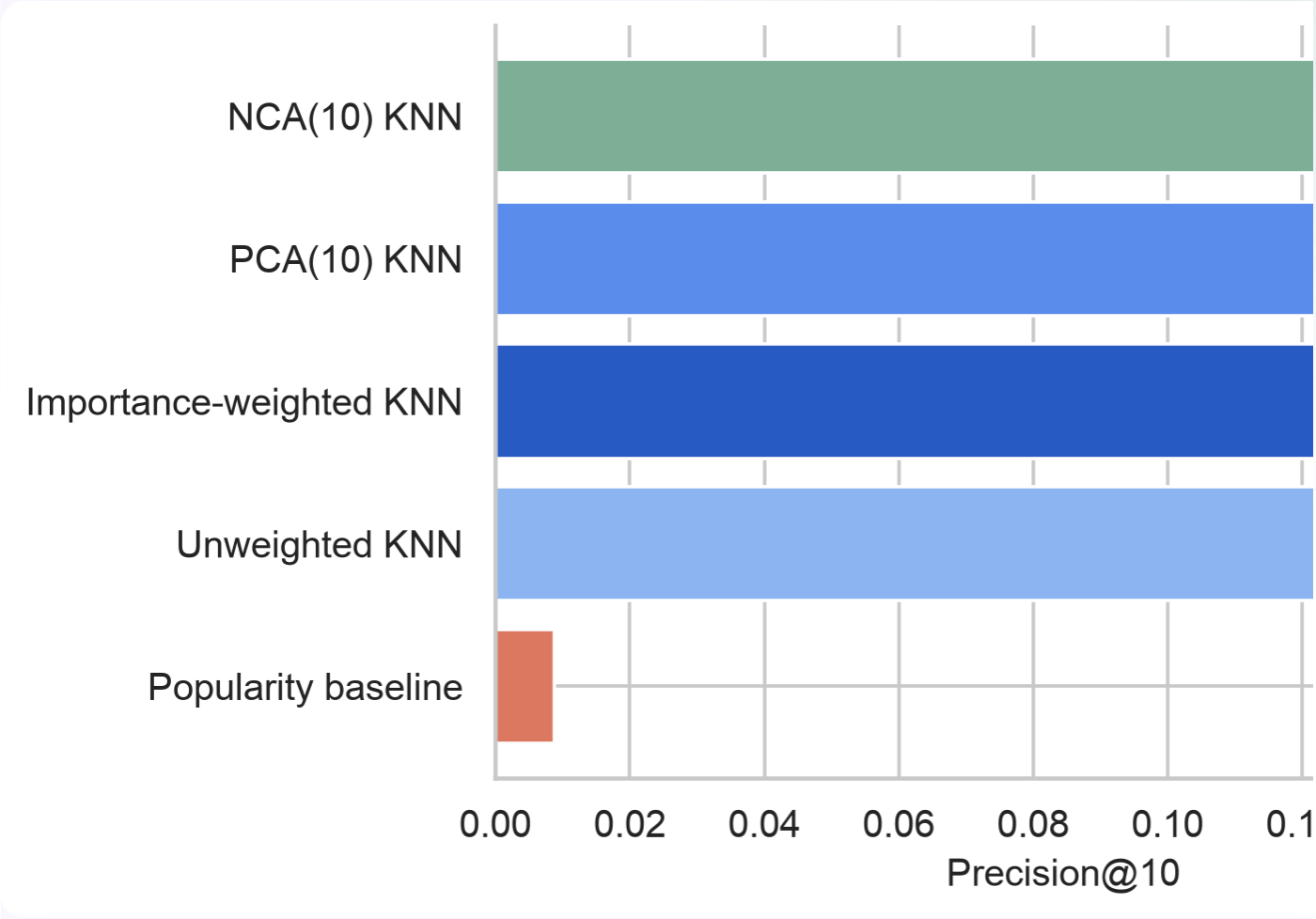
There's no click-through or replay data here, so I needed a different way to check whether the recommendations are any good. I used **precision@k**: hold out a set of tracks, recommend `k` tracks for each from the remaining catalog, and measure what fraction of those recommendations share the query track's true genre. Genre is an imperfect proxy for "actually sounds alike" — two tracks in different genres can sound more similar than two tracks stuck with the same label — but it's the standard fallback when no real feedback signal exists, and it's measurable, which is what I actually need here.

▶ Code

	approach	precision_at_10
0	Popularity baseline	0.008808
1	Unweighted KNN	0.140771
2	Importance-weighted KNN	0.160911

	approach	precision_at_10
3	PCA(10) KNN	0.137506
4	NCA(10) KNN	0.152003

▶ Code



This chart is the payoff of the whole evaluation exercise: every method beats the popularity baseline by a wide margin, which tells me the audio features genuinely carry usable genre signal rather than noise. Importance-weighted KNN came out on top of the five — the more “principled” methods (PCA, NCA) didn’t beat this simpler heuristic here, which genuinely surprised me going in, since NCA is literally optimizing for the metric being measured. That’s a result only knowable by measuring it directly, not by assuming a fancier-sounding technique automatically wins — PCA barely moved the needle (consistent with there being little redundancy to remove in only ~25 features), and NCA improved on plain KNN but still fell short of the importance-weighted approach, plausibly handicapped by having to learn its projection from a 5,000-row subsample rather than the full catalog.

2.3.3 Is exact-genre matching too strict?

Something about the precision@10 numbers above bothered me, so I checked it directly: several of this dataset's 113 genre tags turned out to be near-synonyms applied to the *same physical track* — 24,259 of the raw 114,000 rows are exact-duplicate `track_id`s carrying a different genre tag each time. That means exact-genre precision@k counts a recommendation landing on a near-synonym as a complete miss, which felt like it might be understating how good the recommender actually is. `genre_families.py` groups the 113 tags into 13 broader families (plus a `mood_context` bucket for tags like `chill`/`sad`/`study` that are playlist moods, not musical genres) so I could see how much of that gap is really just near-synonym mismatches.

▶ Code

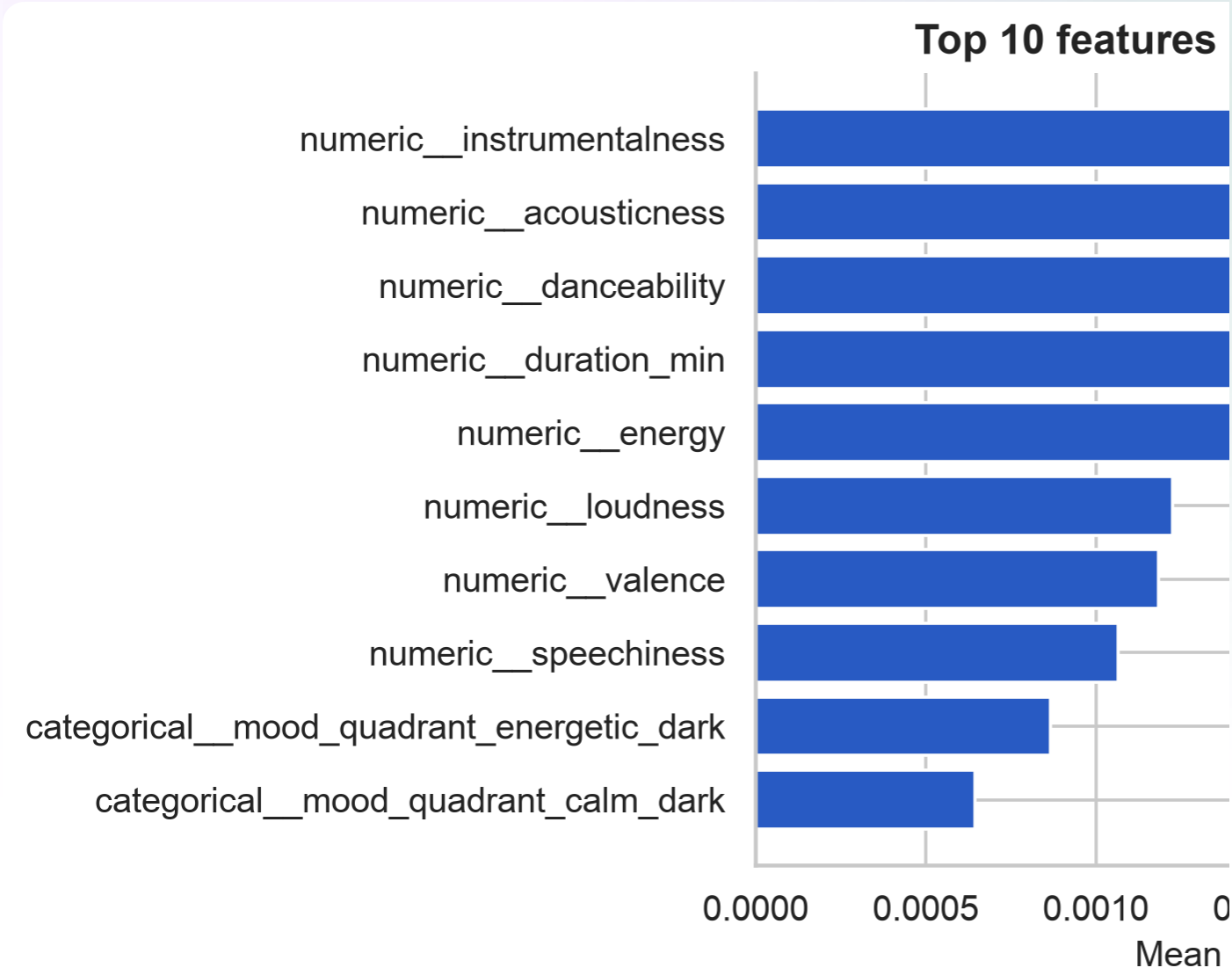
granularity approach	Exact genre	Genre family
Importance-weighted KNN	0.160911	0.331155
Popularity baseline	0.008808	0.122625

My first reaction to this table was that the ~14x jump in the popularity baseline's score might undercut the earlier result — but working through the ratios reassured me. The popularity baseline jumps ~14x at the looser genre-family granularity, while the recommender only roughly doubles — so the recommender's advantage over the naive baseline is actually *larger* in relative terms at the strict exact-genre level (~18x) than at the family level (~2.7x). Coarser bins flatter every method's absolute score, including pure guessing, which is exactly why a bigger number at the family level isn't the more honest one to lead with. What holds up across both framings — and what I actually care about — is that the recommender clearly and consistently beats the baseline either way, just by a different margin depending on how strictly "correct" is defined.

2.4 Interpretability: what separates genres?

The genre classifier exists specifically to produce the importance weights above and to be explained — not to be a high-scoring classifier in its own right (114 genres is a genuinely hard multiclass problem, and I didn't expect it to score well as a classifier on its own terms). I used **SHAP** (SHapley Additive exPlanations) to see which features it actually leans on, rather than take the feature-importance weights on faith.

▶ Code



`instrumentalness` and `acousticness` come out on top, ahead of `danceability` and `duration_min`. That lines up with how I'd describe genres myself — "acoustic" vs. "electronic," "instrumental" vs. "vocal-heavy" are some of the first things I'd listen for — which gives me some confidence that the classifier (and by extension the importance-weighted distance metric it feeds) is picking up on musically real distinctions, not some quirk of the data.

3 Part II — Collaborative Filtering (KKBox)

For the second half of this project I wanted to try the other kind of recommendation problem — one built entirely on real user behavior instead of item properties. The task: given a real member's listening history and a specific song, predict whether they'll listen to it again within a month of first hearing it. I built this on Kaggle's real WSDM — KKBox's Music Recommendation Challenge (2018). Unlike Part I, this needs no audio content at all — the whole task is defined by real user/song

interaction history — and it's scored by AUC on a real Kaggle leaderboard, which meant I could check my results against a public benchmark rather than just my own intuition.

3.1 Data

Four tables, verified directly against the real downloaded files:

- `train.csv` / `test.csv` (7,377,418 / 2,556,791 rows): one row per listening event — `msno` (member), `song_id`, three context columns (`source_system_tab`, `source_screen_name`, `source_type`), and (train only) `target`.
- `songs.csv` (2,296,834 rows): `song_length`, `genre_ids`, `artist_name`, `composer`, `lyricist`, `language`.
- `members.csv` (34,403 rows): `city`, `bd` (self-reported age), `gender`, `registered_via`, `registration_init_time`, `expiration_date`.
- `song_extra_info.csv` (2,296,870 rows): `name`, `isrc`.

▶ Code

```
target balance (sample): 0.506
```

	count	unique	top	freq	mean
msno	50000	16634	o+5RNlSWrzvrphgBNGIo1FLkGxBgylCns6qXj3nS7Pk=	46	NaN
song_id	50000	20987	reXuGcEWDDCnL0K3Th//3DFG4S1ACSpJMzA+CFipo1g=	107	NaN
source_system_tab	49815	8	my library	25012	NaN
source_screen_name	47204	19	Local playlist more	21882	NaN
source_type	49848	12	local-library	15343	NaN
target	50000.0	NaN	NaN	NaN	0.506
song_length	50000.0	NaN	NaN	NaN	2457
genre_ids	49200	220	465	25131	NaN
artist_name	50000	5532	Various Artists	1914	NaN
composer	38785	7969	周杰倫	1385	NaN

Verified directly on the full 7.4M-row `train.csv`: `target` is close to balanced (50.4% replay / 49.6% no-replay). This was a genuine relief to confirm — it means no resampling technique (ADASYN etc.) is needed, and it means the AUC metric introduced below isn't at risk of being distorted by one class dominating the way plain accuracy would be.

3.1.1 Real data-quality issues, found by checking rather than assuming

I almost trusted these columns at face value; checking them directly told a different story, and each finding fed a specific decision in `features.py`:

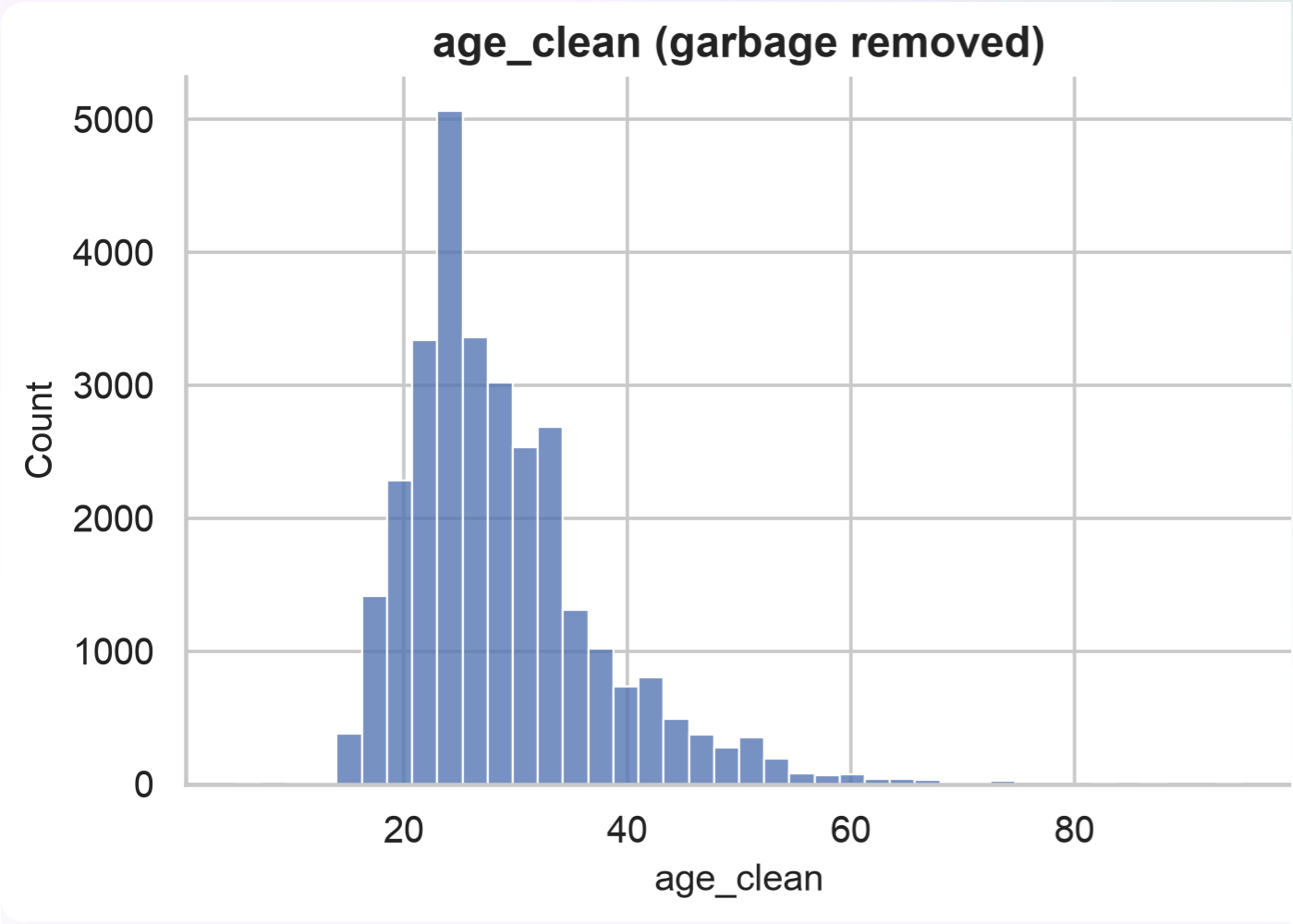
- `bd` **(age) is garbage for ~58% of members** — 0 (the unset default), negative values, and a maximum of **1051**. That's not a typo I can round off; it's a sign the field was never validated on the way in. Anything outside 5-100 becomes `NaN` rather than a trusted age, so the model gets an honest "missing" instead of a number known to be wrong.
- `gender` **missing for ~58%**, `composer` **missing for ~47%**, `lyricist` **missing for ~85%** — with that much missingness, there isn't enough signal in the actual names to model directly, so all three are used only as coarse presence signals.
- `genre_ids` **is multi-valued for ~7.9%** of songs (pipe-separated) — counted, not just checked for presence, since that's real information worth keeping.
- `artist_name` **has 222,363 distinct values** — far too many to one-hot encode without producing an enormous, mostly-empty matrix; replaced with a frequency encoding instead (see Methodology).

3.2 Feature engineering

`src/recommendation_showcase/collaborative/features.py` is the single source of truth for turning these raw, messy columns into model-ready features:

- `InteractionCountEncoder`: `song_id`, `msno` (user), and `artist_name` are all identity columns too high-cardinality to one-hot encode, so instead this learns `song_popularity`, `user_activity_count`, and `artist_song_count` — how often each song/user/artist appears — from the *training* interactions only (a song/user/artist never seen in training gets 0, which is itself meaningful signal — a song nobody's played yet — not a gap to fill). This is the collaborative-filtering analogue of Part I's evaluation discipline: learn a lookup from training data only, apply it unchanged to any future row, so the model never peeks at counts it wouldn't actually have at prediction time.
- `FeatureEngineer`: `member_tenure_days` (registration to expiration), `age_clean` (the garbage-`bd` fix above), `genre_count` / `primary_genre` (from pipe-separated `genre_ids`), `composer_count` / `has_composer` / `has_lyricist`, and `isrc_country` / `isrc_year` (parsed from `song_extra_info.isrc`, with a century pivot for the 2-digit year).
- `RareCategoryGrouper`: a standard technique for long-tailed categorical variables, applied here to `primary_genre` (182 distinct values).

▶ Code



`age_clean` now looks like an actual age distribution, clustered in the 20s and 30s, instead of the spike-at-zero mess `bd` was — a visible payoff from the cleanup above. `song_popularity` is heavily right-skewed even on a log axis: most songs get played by only a handful of listeners, while a small number rack up thousands of interactions. That long tail is exactly the kind of signal a frequency encoding is built to capture, and exactly what a one-hot encoding on `song_id` would have thrown away.

3.3 Methodology

3.3.1 Why one train/holdout split, not k-fold cross-validation

At 7.4M rows, a single fit of a gradient-boosted model already takes real time (verified directly: LightGBM, ~91 seconds for the full join + feature engineering + fit), so I had to weigh whether k-fold cross-validation was worth its cost here. Cross-validation earns its keep when there's a real

leakage risk a single split might hide — chronological structure, or the same identity appearing in both halves — and unlike Part I, I don't see either risk in this data. So one honest 80/20 split, not 5-fold CV, is the tractable and still-honest choice given the training cost.

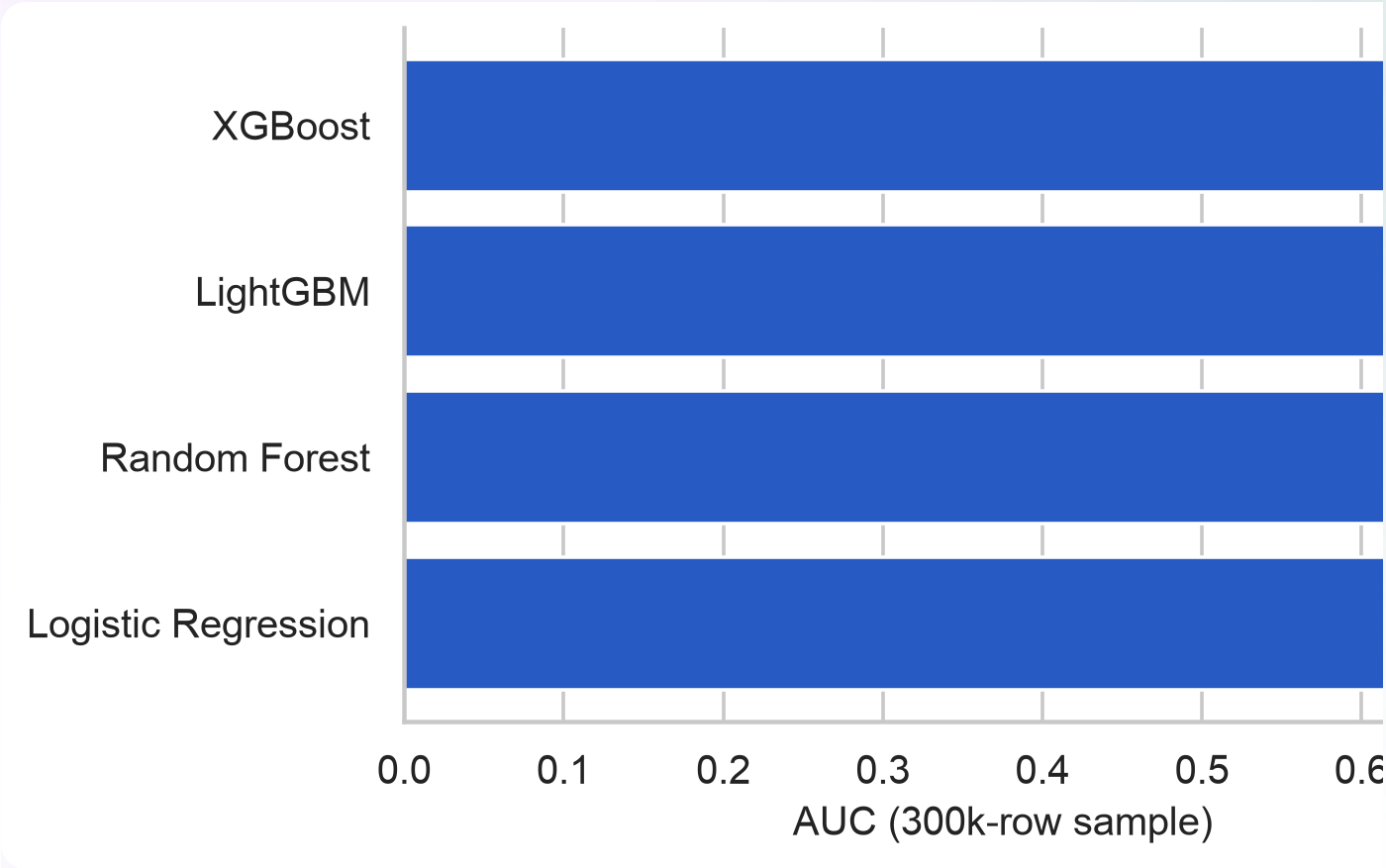
3.3.2 Model comparison

AUC (area under the ROC curve) is the probability that the model ranks a randomly chosen "replayed" interaction above a randomly chosen "not replayed" one — 0.5 is a coin flip, 1.0 is perfect separation. I trust it here specifically because the target is close to balanced (see Data above), so it isn't being distorted by one class dominating the score the way plain accuracy would be.

▶ Code

	model	auc
3	XGBoost	0.714414
2	LightGBM	0.714181
1	Random Forest	0.694301
0	Logistic Regression	0.687687

▶ Code



The two gradient-boosted models (XGBoost, LightGBM) clear Random Forest and Logistic Regression by a real margin here, though XGBoost and LightGBM are close enough to each other that I wouldn't call the gap meaningful. What told me more than the AUC column was fit time: even on this small 300k-row sample, Random Forest already took noticeably longer than either boosted model — an early hint of the scalability problem below. Random Forest is excluded from `scripts/train_collaborative.py`'s default by design — verified directly that `RandomForestClassifier(n_estimators=200, max_depth=12)` didn't finish fitting on the full 5.9M-row training split even after 500 seconds, while LightGBM finished the equivalent job in 91 seconds. The sweep above uses a 300k-row sample specifically so Random Forest *can* be included for a fair comparison point — not because it would scale to the real dataset. A model that wins on 300k rows is worthless in production if it can't finish training on the real dataset in reasonable time.

3.3.3 Full-scale result

▶ Code

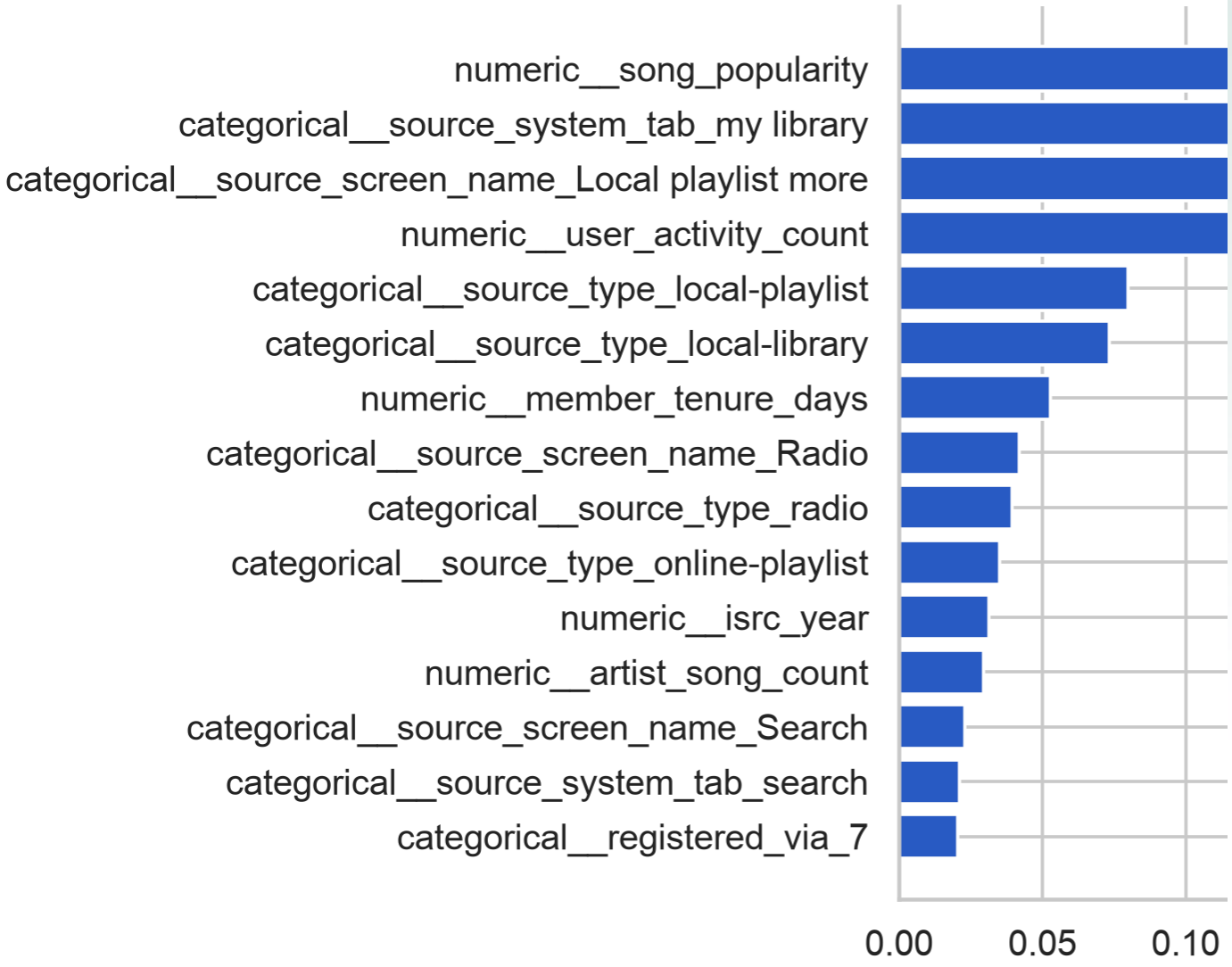
```
LightGBM, full 7.4M-row panel: AUC = 0.7253
  (trained on 5,901,934 rows, evaluated on 1,475,484)
```

The full-scale AUC comes out noticeably higher than the 300k-row sample's LightGBM result — a sign that more training data is still buying real improvement at this scale, not diminishing returns, and that training on the full panel (rather than settling for the cheaper sample) was worth the extra compute time.

3.4 Interpretability

I wanted to check *why* the model predicts a replay, not just trust the AUC number, so I used **SHAP** here too — the same approach as Part I's genre classifier, but now on a binary yes/no prediction instead of a 114-way multiclass one.

▶ Code



`song_popularity` and `user_activity_count` rank at or near the top, which is genuinely the result I was hoping for rather than a coincidence: it directly validates the central design choice of this half of the project — the collaborative-filtering signal `InteractionCountEncoder` was built specifically to capture is what the model actually leans on, not a plausible-sounding feature buried under everything else. If member-attribute features like `age_clean` had dominated instead, that would have told me the feature engineering wasn't capturing the interaction signal it was designed to.

4 Head-to-Head Comparison

Putting these two builds side by side is the part of this project I was most curious about going in. Both approaches solve "recommend a song," but they aren't interchangeable, and I didn't want to

pretend one number being bigger than the other means anything — the honest comparison is about what each technique needs and what it's actually good for.

▶ Code

	Aspect	Content-based (Spotify)	Collaborative filtering (KKBox)
0	Data required	None — just item features	Real user/item interaction history
1	Cold-start (brand-new item/user)	New tracks work immediately (their own feature…	New songs/users get 0 interaction counts — a r…
2	Evaluation metric	precision@k (genre-sharing proxy)	AUC (real Kaggle leaderboard)
3	Headline result	0.161 precision@10 (exact genre), 0.331 (genre…	0.7253 AUC, full 7.4M-row training set
4	Explains recommendations by	Feature similarity (radar chart of audio prope…	Interaction frequency + member/song attributes…
5	Dataset scale	~90k tracks (deduplicated)	7.4M interactions, 2.3M songs, 34k members

Neither number in the "Headline result" row is comparable to the other in any direct sense — precision@10 and AUC measure different things on different tasks, and I want to be explicit that reading one as "better" than the other would be a mistake. What *is* comparable, and what I actually take away from this table, is the discipline behind each: both halves of this project hold out data honestly, compare against a naive baseline before claiming a win, and measure scalability/cost tradeoffs directly rather than assuming them (NCA's memory limit in Part I; Random Forest's full-scale timeout in Part II). The cold-start row is the one I'd flag as the most practically important difference — it's the kind of thing that would decide which technique I'd actually reach for on a real product with both brand-new items and an established user base.

5 Discussion

- **Part I's importance-weighted KNN won** its own comparison, beating unweighted KNN, PCA, and NCA — the two more mathematically sophisticated methods didn't pay off here, which genuinely surprised me going in. The finding I'd actually report from this: a fancier-sounding technique isn't automatically better, and the only way I know that is by measuring it under the same evaluation as everything else, not by trusting which one sounds more advanced.
- **Part I's exact-genre precision@k understates recommendation quality, but not by as much as it might look** — the naive baseline's score grows even faster at the looser genre-family granularity, so the recommender's real edge is clearest at the strict, exact-genre level. This

mattered to me because my first instinct on seeing the family-level numbers was that they looked more encouraging — working through the ratios showed the opposite was true.

- **Part II's real, measured scalability finding**: Random Forest doesn't finish fitting on the full KKBox training set in a reasonable time, while LightGBM does the same job in under two minutes. That's worth knowing before assuming a bigger, ostensibly-more-powerful ensemble is always the safer default — the model that wins a small-sample sweep isn't automatically the model that ships.
- A limitation worth naming for both, honestly: Part I's genre-sharing is an imperfect proxy for "sounds alike"; Part II's single train/holdout split (not k-fold CV) is an honest tradeoff for training cost, but one fold's variance is real, and I'd want to see that variance quantified before trusting the 0.7253 AUC to four decimal places in a production setting.

6 Conclusion

Content-based and collaborative filtering solve the same stated problem — "recommend something" — with almost entirely non-overlapping requirements, evaluation strategies, and failure modes. Building both on real data, under the same standards of honest evaluation and measured (not assumed) engineering tradeoffs, made the actual difference between the two techniques legible to me in a way that reading about either one in the abstract never would have. If I had to summarize what changed in how I think about recommendation after this project: it's not "which technique is better," it's "what data do I actually have, and what does that data force me to give up."